Trabajo final: KNN vs. LSTM

Métodos para el Análisis de Datos I

Pilar Maiztegui

Universidad Nacional del Sur

2026

Presentación del problema.

Como las tecnologías digitales se volvieron fundamentales en varios aspectos de nuestras vidas, nos resulta importante detectar ciberataques para proteger nuestra información.
Los métodos de aprendizaje automático nos ayudan a predecir estas intrusiones en red, detectando patrones difíciles de identificar.

Objetivo

Comparar KNN y LSTM para la detección y clasificación de estas amenazas.

Presentación del problema.

Trabajamos con el artículo *Deep Learning vs. Machine Learning for Intrusion Detection in Computer Networks: A Comparative Study*, Ali et. al (2025) que utiliza el dataset CICIDS-2017.

¿Por qué comparar KNN y LSTM?

KNN es un modelo clásico y LSTM un modelo de aprendizaje profundo. Nos interesa comparar:

- Métricas.
- Como se comportan con este dataset.
- Cuál es mejor para el contexto del problema.

Dataset.

El CICIDS-2017 es un dataset creado por el Canadian Institute of Cybersecurity para la detección de intrusiones. Fue generado durante 5 días, capturando tráfico de red (normal y amenazas).

Tipos de ataques:

- Web Attack (Brute Force, XSS, Sql Injection)
- DoS (Hulk, slowloris, Slowhttptest, GoldenEye)
- DDoS
- PortScan
- FTP-Patator y SSH-Patator
- Bot
- Infiltration
- Heartbleed

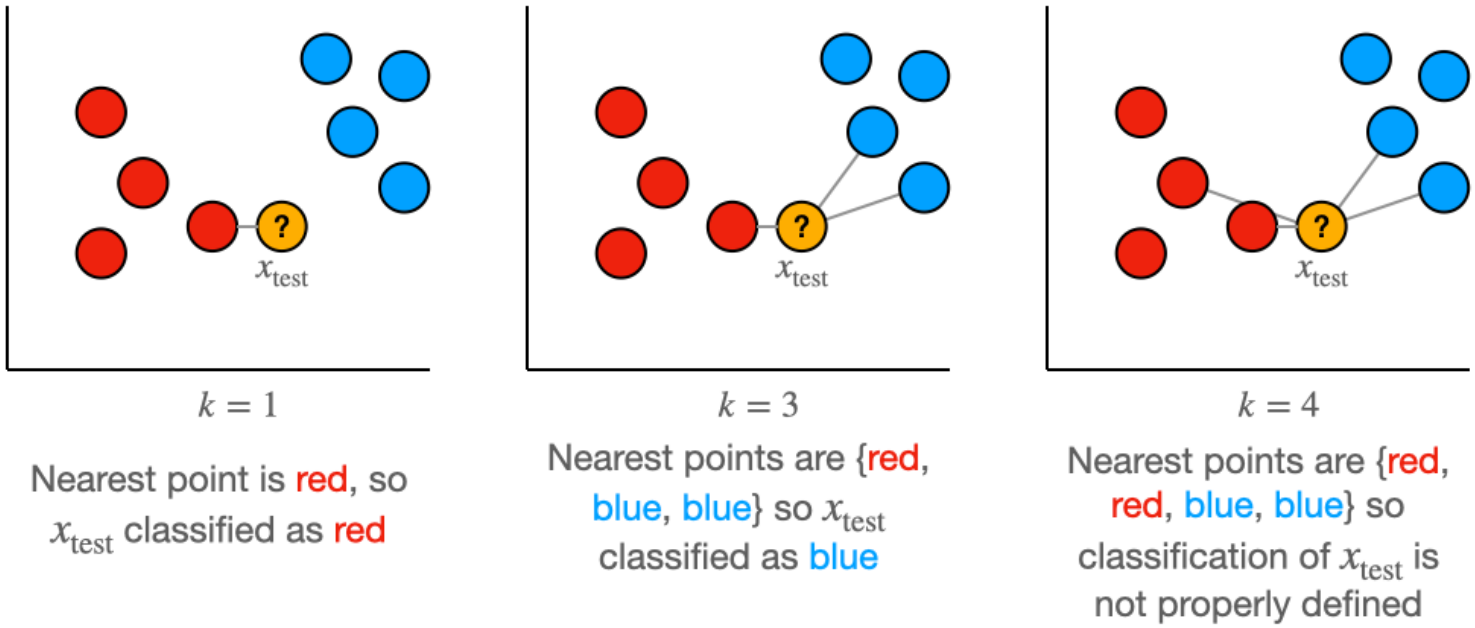
Dataset.

Durante el preprocesamiento de los datos, eliminamos duplicados, completamos datos faltantes y agrupamos tipos de ataques para simplificar la clasificación.
A continuación se muestra la distribución de clases antes y después de aplicar SMOTE:

Clase	**Antes de SMOTE**	**Después de SMOTE**
BENIGN	1.676.045	1.676.045
DoS	154.996	1.676.045
DDoS	102.411	1.676.045
PortScan	72.555	1.676.045
Other	7.358	1.676.045
Web Attack	1.714	1.676.045
Bot	1.559	1.676.045

KNN: K-Nearest Neighbors - Marco teórico

- Idea general: KNN es un algoritmo de clasificación que asume que los puntos cercanos pertenecen a una misma clase. Si queremos clasificar un nuevo dato x_0, este algoritmo busca sus k vecinos más cercanos y le asigna la clase mayoritaria de ellos.



KNN: K-Nearest Neighbors - Marco teórico.

Cosas a tener en cuenta al trabajar con el modelo:

- Por simplicidad, usamos la distancia Euclídea, aunque se podría optar el uso de otras, como la Manhattan o la Minkowski.
- Se recomienda estandarizar cada variable para que tengan media 0 y varianza 1, lo que evita que las medidas de mayores escalas tengan mayor influencia sobre las distancias.
- Elección del hiperparámetro: el valor de k determina la cantidad de vecinos a considerar para clasificar. Se suele probar con números impares hasta el 15 o 17.
  - Si $k = 1$ la frontera es irregular. Bajo sesgo pero alta varianza.
  - Si k demasiado grande, la frontera es suave y el modelo ignora detalles locales, aumentando el sesgo.

KNN: K-Nearest Neighbors - Marco teórico.

Ventajas

- Simplicidad: fácil de entender e implementar.
- Rápido entrenamiento.
- Depende de un solo hiperparámetro.

Limitaciones

- Maldición de la dimensión: en espacios de alta dimensión, todos los puntos parecen estar a casi la misma distancia. Se recomienda eliminar variables redundantes.
- Lento para predecir: tiene que comparar todas las distancias.

KNN: K-Nearest Neighbors - Aplicación.

- Elección del k: Recordemos que teniamos 11 millones de filas. Elegimos tomar una muestra de 100.000 observaciones y hacer validación cruzada sobre ella para elegir el k.
  Probamos con valores impares de k entre 5 y 11, y nos quedamos con el que mejor F1 macro promedio nos devolvía. Esto nos dió:

k	**F1 macro promedio**
5	0.9893
7	0.9881
9	0.9870
11	0.9860

Por lo tanto, elegimos $k = 5$

LSTM: Long Short Term Memory - Marco teórico.

- LSTM es un tipo de red neuronal recurrente (RNN) con compuertas. Estas se basan en crear caminos en el tiempo conectando con pasos que pueden cambiar en cada caso.
- Tiene una célula de memoria con una recurrencia interna. Cada célula tiene la misma cantidad de entradas y salidas que la red, pero tiene más parámetros y un sistema de compuertas que controlan el flujo de información.
- Cada célula tiene una puerta de entrada que controla el ingreso de nueva información y determina si es relevante para almacenar; una puerta de olvido que decide que información se puede descartar;y una puerta de salida que controla qué parte de la célula se usa para generar la salida en el paso actual.

LSTM: Long Short Term Memory - Marco teórico.

Ventajas

- Memoria a largo plazo.
- Funciona muy bien para datos que dependen del tiempo.
- Rápida predicción.

Limitaciones

- Entrenamiento lento.
- Mayor uso de memoria.
- Mayor costo computacional debido a la estructura interna.

LSTM: Long Short Term Memory - Aplicación.

- Trabajamos con la misma muestra de 100.000 observaciones que usamos para KNN.
- Primero, transformamos las etiquetas de texto a números para que la red las pueda interpretar:

```
0 -> BENIGN
1 -> Bot
2 -> DDoS
3 -> DoS
4 -> Other
5 -> PortScan
6 -> Web Attack
```

LSTM: Long Short Term Memory - Aplicación.

- Separamos la muestra en 80.000 datos para entrenamiento y 20.000 para validación.
- Tuvimos que reformar a X (features) porque necesitabamos datos tridimensionales.
- Tuvimos que elegir cuantas pasadas (épocas) por la red haciamos durante el entrenamiento. Esto es importante para no quedarse con un modelo que no haya aprendido lo suficiente o que sobreajuste demasiado. Lo hicimos mediante EarlyStopping, monitoreando el error sobre el error de validación, con un máximo de 30 épocas. Si el error no mejoraba durante dos épocas consecutivas del entrenamiento, éste se detenía y se recuperaban los pesos de la época con mejor validación.

LSTM: Long Short Term Memory - Aplicación.

```
Epoch 1/15
157/157 ━━━━━━━━━━━━━━━━━━━━ 48s 305ms/step - accuracy: 0.5430 - loss: 1.2849 - val_accuracy: 0.7484 - val_loss: 0.7284
Epoch 2/15
157/157 ━━━━━━━━━━━━━━━━━━━━ 53s 335ms/step - accuracy: 0.8325 - loss: 0.5652 - val_accuracy: 0.8561 - val_loss: 0.4813
Epoch 3/15
157/157 ━━━━━━━━━━━━━━━━━━━━ 48s 303ms/step - accuracy: 0.8759 - loss: 0.4135 - val_accuracy: 0.8980 - val_loss: 0.3218
Epoch 4/15
157/157 ━━━━━━━━━━━━━━━━━━━━ 52s 330ms/step - accuracy: 0.8967 - loss: 0.3424 - val_accuracy: 0.9191 - val_loss: 0.2778
Epoch 5/15
157/157 ━━━━━━━━━━━━━━━━━━━━ 48s 308ms/step - accuracy: 0.9173 - loss: 0.2760 - val_accuracy: 0.9289 - val_loss: 0.2391
Epoch 6/15
157/157 ━━━━━━━━━━━━━━━━━━━━ 48s 307ms/step - accuracy: 0.9234 - loss: 0.2548 - val_accuracy: 0.9348 - val_loss: 0.2158
Epoch 7/15
157/157 ━━━━━━━━━━━━━━━━━━━━ 51s 324ms/step - accuracy: 0.9107 - loss: 0.2970 - val_accuracy: 0.9294 - val_loss: 0.2094
Epoch 8/15
157/157 ━━━━━━━━━━━━━━━━━━━━ 64s 410ms/step - accuracy: 0.9335 - loss: 0.2194 - val_accuracy: 0.9464 - val_loss: 0.1827
Epoch 9/15
157/157 ━━━━━━━━━━━━━━━━━━━━ 63s 400ms/step - accuracy: 0.9393 - loss: 0.2001 - val_accuracy: 0.9481 - val_loss: 0.1722
Epoch 10/15
157/157 ━━━━━━━━━━━━━━━━━━━━ 60s 382ms/step - accuracy: 0.9423 - loss: 0.1910 - val_accuracy: 0.9481 - val_loss: 0.1667
Epoch 11/15
157/157 ━━━━━━━━━━━━━━━━━━━━ 61s 389ms/step - accuracy: 0.9466 - loss: 0.1764 - val_accuracy: 0.9527 - val_loss: 0.1563
Epoch 12/15
157/157 ━━━━━━━━━━━━━━━━━━━━ 51s 323ms/step - accuracy: 0.9213 - loss: 0.2578 - val_accuracy: 0.9489 - val_loss: 0.1667
Epoch 13/15
157/157 ━━━━━━━━━━━━━━━━━━━━ 49s 315ms/step - accuracy: 0.9477 - loss: 0.1737 - val_accuracy: 0.9520 - val_loss: 0.1480
Epoch 14/15
157/157 ━━━━━━━━━━━━━━━━━━━━ 49s 309ms/step - accuracy: 0.9505 - loss: 0.1617 - val_accuracy: 0.9563 - val_loss: 0.1418
Epoch 15/15
157/157 ━━━━━━━━━━━━━━━━━━━━ 51s 324ms/step - accuracy: 0.9541 - loss: 0.1533 - val_accuracy: 0.9581 - val_loss: 0.1370
Epoch 16/30
157/157 ━━━━━━━━━━━━━━━━━━━━ 49s 315ms/step - accuracy: 0.9561 - loss: 0.1460 - val_accuracy: 0.9613 - val_loss: 0.1252
Epoch 17/30
157/157 ━━━━━━━━━━━━━━━━━━━━ 51s 325ms/step - accuracy: 0.9244 - loss: 0.2578 - val_accuracy: 0.9097 - val_loss: 0.2785
Epoch 18/30
157/157 ━━━━━━━━━━━━━━━━━━━━ 51s 327ms/step - accuracy: 0.9406 - loss: 0.1998 - val_accuracy: 0.9424 - val_loss: 0.1675
```

Figure: EarlyStopping

Así, decidimos que lo mejor era quedarnos con 16 épocas para el entrenamiento.

Métricas.

¿Qué significa cada métrica?
Tenemos los verdaderos positivos (VP), los verdaderos negativos(VN), los falsos positivos (FP) y los falsos negativos (FN).

- *Accuracy:* Mide la proporción de predicciones correctas sobre el total de predicciones. Es engañosa para clases desbalanceadas.
- *Precisión:* Nos dice qué proporción de las predicciones del modelo son correctas.
- *Recall:* Indica la proporción de los casos positivos reales que detecta el modelo.
- *F1-Score:* Relaciona la precisión y el recall.
  - *F1 macro:* Calcula el F1 de cada clase y los promedia, lo que permite detectar si el modelo funciona mal en algunas clases.
  - *F1 ponderado:* Calcula el F1 de cada clase pero promediamos dandole más peso a las clases más grandes.

Métricas.

Fórmulas

- Accuracy$= \frac{VP+VN}{VP+VN+FP+FN}$
- Precisión$= \frac{VP}{VP+FP}$
- Recall$= \frac{VP}{VP+FN}$
- F1$= 2 \cdot \frac{\text{Precisión} \cdot \text{Recall}}{\text{Precisión}+\text{Recall}}$
- $F1_{\text{macro}} = \frac{\sum_{i=1}^{C} F1_i}{C}$ donde C es la cantidad de clases.
- $F1_{\text{ponderado}} = \frac{\sum_{i=1}^{C} n_i F1_i}{\sum_{i=1}^{C} n_i}$, donde n_i la cantidad de observaciones para la clase C_i

Resultados.

Nos interesa comparar nuestros resultados luego de predecir con los hallados en *Ali et. Al*:

Métrica	**KNN**	**KNN Ali**	**LSTM**	**LSTM Ali**
Accuracy	96.41%	99.36%	86.76%	98%
F1 Macro	70.92%	97.62%	57.24%	84%
F1 Ponderado	97.12%	99%	89.75%	98%

Notamos que nuestras métricas son más bajas. ¿Qué pudo pasar?
Nosotros elegimos entrenar con una muestra de 100.000 datos con clases balanceadas mediante SMOTE y luego predijimos sobre el conjunto original, lo cual puede explicar las diferencias en las métricas.
En particular, las clases más pequeñas fueron las más difíciles de clasificar, lo que explica el F1 macro bajo.

¿Qué método gana en este caso?

En nuestro experimento, al igual que en el paper de referencia, KNN obtiene mejores resultados según las métricas comparadas. Es por eso que KNN es la alternativa conveniente para este análisis.
LSTM funciona en particular muy bien para analizar patrones temporales. Sin embargo, las observaciones de CICIDS-2017 son características ya calculadas del tráfico y no una verdadera secuencia temporal.

¿Cuándo gana LSTM? - Ejemplo.

Paper de referencia:

"A comparative study of machine learning algorithms for fall detection in technology-based healthcare system: analyzing SVM, KNN, decision tree, random forest, LSTM, and CNN", (Afuan e Isnanto, 2025)

El objetivo está puesto en detectar caídas, pensando en el cuidado de los adultos mayores.
La dificultad está en encontrar un algoritmo eficiente que diferencié entre caídas reales y actividades normales.

¿Cuándo gana LSTM? - Ejemplo.

Afuan e Isnanto trabajaron con el dataset de Kaggle Smartphone Human Fall Dataset, que incluye variables como aceleración máxima, velocidad angular, velocidad angular luego de la caída, aceleración lineal luego de la caída, entre otras.
Luego de ser preprocesado y entrenado, al momento de predecir observaron:

Algoritmo	**Accuracy**	**Recall**	**Precisión**	**F1-score**
KNN	87%	80%	85%	81%
LSTM	92%	92%	91%	92%

¿Cuándo gana LSTM? - Ejemplo.

¿Por qué es mejor LSTM?

Los datos provienen de sensores que registran movimientos de una persona, y las caídas pueden incluir cambios repentinos de aceleración y velocidad. Estas combinaciones pueden crear patrones complejos, más díficiles de reconocer para algoritmos como KNN. Pero favorece a los modelos de aprendizaje profundo como LSTM, que además, si recibiera las mediciones en orden temporal, podría aprender como evoluciona el movimiento antes, durante y después de la caída

KNN funciona mejor para clasificar observaciones que se distinguen por sus características. En este caso, una caída podría tener un valor parecido a una actividad normal en alguna variable, aunque el patrón sea distinto.

Bibliografía.

- Afuan, L., & Isnanto, R. R. (2025). *A comparative study of machine learning algorithms for fall detection in technology-based healthcare system: analyzing SVM, KNN, decision tree, random forest, LSTM, and CNN*. E3S Web of Conferences (Vol. 605), 03051.
- Ali, M. L., Thakur, K., Schmeelk, S., Debello, J., & Dragos, D. (2025). *Deep Learning vs. Machine Learning for Intrusion Detection in Computer Networks: A Comparative Study*. Applied Sciences, 1903.
- Goodfellow, I., Bengio, Y., & Courville, A. (2015). *Deep Learning*. MIT Press.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer.